
INDUSTRIAL IoT BASED MACHINE FAULT DETECTION SYSTEM



A

Project Report

Submitted in partial fulfillment of the requirements for the award of degree of

BACHELOR OF TECHNOLOGY

in

ELECTRONICS AND COMMUNICATION ENGINEERING

Submitted by

Group No. 02

Aarushi Bhatnagar (2316651)

Agrima Gusain (2316652)

Akanksha Patel (2316653)

Rudrakshi Upadhyay (2316726)

Saloni Mishra (2316728)

Under the guidance of

CMS Negi

Assistant Professor

Department of Physical Sciences

Banasthali Vidyapith

Rajasthan

DEPARTMENT OF PHYSICAL SCIENCES

BANASTHALI VIDYAPITH

Rajasthan, India

2026

Candidates' Declaration

We hereby declare that the work, which is being presented in the Project, entitled **“Industrial IoT Based Fault Detection and Prevention System”** in partial fulfillment for the award of Degree of “*Bachelor* of Technology” in Department of Physical Sciences with Specialization in Electronics and Communication Engineering, **and submitted to the Department of Physical Sciences, Banasthali Vidyapith, Rajasthan** is a record of our own investigations carried under the Guidance of Dr. CMS Negi, Department of Electronics, Banasthali Vidyapith.

I have not submitted the matter presented in this report anywhere for the award of any other Degree.

Aarushi Bhatnagar (BTBTE23087)
Agrima Gusain (BTBTE23088)
Akanksha Patel (BTBTE23066)
Rudrakshi Upadhyay (BTBTE23028)
Saloni Mishra (BTBTE23144)

Banasthali Vidyapith, Rajasthan

CERTIFICATE

This is to certify that Aarushi Bhatnagar, Agrima Gusain, Akanksha Patel, Rudrakshi Upadhyay, Saloni Mishra of VI Semester, B.Tech. (Electronics & Communication Engineering) 2026-27, have presented a project titled “Industrial IoT Based Fault Detection and Prevention System” in partial fulfillment for the award of the degree of Bachelor of Technology under Banasthali Vidyapith, Rajasthan.

Date: 27/3/2026

Dr. CMS Negi

Mentor

CONTENTS

Title	Page No.
1. Abstract	05
2. Introduction	06
3. Circuit Diagram	10
4. Component List	17
5. Component Description	19
6. Costing of Project	23
7. Advantages and Disadvantages	25
8. Conclusion	27
9. Future Scope	28
10. Appendices	30
11. References	36

Abstract

This report presents the design and implementation of an Industrial Internet of Things (IIoT) based machine fault detection and prevention system. The system is developed to monitor the health of a 5V DC motor in real time by simultaneously measuring three critical parameters: operating current, surface temperature, and structural vibration. The ACS712 20A Hall-effect current sensor measures the current flowing through the motor supply line and produces an analog voltage output, which is digitized by the PCF8591 Analog-to-Digital Converter (ADC) module over the I²C bus before being read by the Raspberry Pi. The DS18B20 digital temperature sensor measures the motor surface temperature and communicates directly with the Raspberry Pi 5 using the 1-Wire protocol. The MPU6050 Inertial Measurement Unit measures three-axis acceleration and gyroscope data to detect abnormal vibration and communicates with the Raspberry Pi 5 over the I²C bus.

All acquired sensor data is processed using a classification model that evaluates the motor's condition and classifies it as either Normal or Jam. Upon detection of a fault, the system activates visual LED indicators and an audible buzzer alarm, and automatically disconnects the motor from its power supply using a 5V single-channel relay module for hardware-level protection. A real-time web-based dashboard serves as a centralized interface for displaying live sensor readings, system status, and logged fault events. This project demonstrates the use of multi-parameter sensing and IoT-based monitoring for assessing the health of an industrial motor.

1. Introduction

1.1 Background and Motivation

The reliability and continuous operation of electrical machines are fundamental to the productivity and safety of modern industrial processes. Electric motors constitute the backbone of industrial automation, powering conveyors, compressors, pumps, fans, and a multitude of manufacturing equipment. According to industry data, motor failures account for a significant proportion of unplanned industrial downtime, resulting in substantial financial losses, safety hazards, and reduced operational efficiency.

Traditional maintenance approaches, such as periodic scheduled maintenance, are often inadequate as they do not account for the actual real-time condition of the machine. Reactive maintenance, responding only after a failure has occurred, leads to costly emergency repairs, production halts, and potential safety incidents. Predictive maintenance, enabled through continuous sensor-based monitoring and intelligent fault detection, represents a paradigm shift that allows maintenance interventions to be planned based on the actual health of the machine rather than on fixed time intervals.

Industrial motor faults rarely occur instantaneously. They typically develop progressively through a combination of electrical and mechanical degradation mechanisms. Electrical faults such as overcurrent and overheating affect the motor's windings and insulation, while mechanical faults such as bearing wear, rotor imbalance, and shaft misalignment manifest as abnormal vibration patterns in the motor structure. Effective fault detection therefore requires the simultaneous monitoring of both electrical parameters and mechanical behaviour to provide a comprehensive assessment of machine health. The rapid advancement of Internet of Things (IoT) technology has made it increasingly feasible to deploy low-cost, networked sensor systems capable of acquiring, processing, and transmitting machine health data in real time, enabling timely fault detection and automated preventive action.

1.2 Problem Statement

In conventional industrial settings, machine faults such as overheating, overloading, bearing failure, and mechanical imbalance develop gradually before manifesting as

complete and often catastrophic failures. The absence of a real-time, automated monitoring mechanism means that incipient faults frequently go undetected until they escalate into serious and costly equipment damage. Furthermore, even when human operators are present, manual inspection cannot provide the continuous, uninterrupted observation necessary to reliably detect transient or progressively developing fault conditions across multiple parameters simultaneously.

Monitoring a single parameter such as current or temperature alone is insufficient for comprehensive fault detection. Mechanical faults such as bearing degradation or rotor imbalance may not produce significant changes in current or temperature in their early stages, yet they generate characteristic abnormal vibration signatures that are detectable well before electrical parameters are affected. This project addresses the problem by designing a system that simultaneously monitors motor current, surface temperature, and structural vibration, compares all parameters against safe operating limits using a trained AI model, alerts operators, and triggers automated protective actions upon confirmation of a fault condition.

1.3 Project Objectives

The specific objectives of this project are as follows:

- To design an embedded multi-parameter sensor interface system capable of simultaneously measuring motor operating current, surface temperature, and structural vibration in real time using the ACS712 20A current sensor, the DS18B20 digital temperature sensor, and the MPU6050 Inertial Measurement Unit respectively.
- To implement analog-to-digital conversion of the ACS712 current sensor's analog output using the PCF8591 ADC module, which communicates with the Raspberry Pi 5 over the I²C bus, compensating for the Raspberry Pi 5's lack of native analog input pins.
- To acquire three-axis acceleration and gyroscope data from the MPU6050 sensor over the I²C bus and process this data to detect abnormal vibration patterns indicative of mechanical faults such as bearing wear, rotor imbalance, or shaft misalignment.

-
- To develop and deploy a trained AI classification model on the Raspberry Pi 5 that evaluates all incoming sensor readings and classifies the motor's operating condition as Normal, Warning, or Fault.
 - To implement a relay-based automatic protection mechanism, wherein a 5V single-channel relay module disconnects the motor from its power supply upon detection of a confirmed fault condition, preventing further machine damage.
 - To generate appropriate visual and audible alert signals through LED indicators and a piezo buzzer, and to log all sensor readings and fault events with timestamps for post-event analysis.
 - To develop a real-time web-based monitoring dashboard that displays live sensor data, system status, and historical fault logs, accessible from any networked device.
 - To evaluate the overall system performance in terms of fault detection accuracy, response time, and operational reliability across all three monitored parameters.

1.4 Scope of the Project

This project focuses on the simultaneous monitoring of three key fault indicators — operating current, motor surface temperature, and structural vibration — as these parameters collectively cover both the electrical and mechanical failure modes most commonly encountered in industrial motor applications.

For current monitoring, the ACS712 20A current sensor is employed. This Hall-effect-based sensor measures the current flowing through the motor supply line and produces an analog output voltage proportional to the instantaneous current. Since this output is an analog signal, it is digitized by the PCF8591 Analog-to-Digital Converter module, which interfaces with the Raspberry Pi 5 over the I²C bus.

For temperature monitoring, the DS18B20 digital temperature sensor is used. The DS18B20 communicates directly with the Raspberry Pi 5 via the 1-Wire digital protocol on a dedicated GPIO pin, providing calibrated 12-bit temperature readings without requiring any analog-to-digital conversion. A 4.7 k Ω pull-up resistor is connected between the data line and the supply rail, as required by the 1-Wire protocol specification.

For vibration monitoring, the MPU6050 Inertial Measurement Unit is employed. The MPU6050 integrates a three-axis MEMS accelerometer and a three-axis MEMS gyroscope in a single package and communicates directly with the Raspberry Pi 5 over the I²C bus. The system uses the MPU6050's accelerometer data to detect abnormal vibration magnitudes associated with mechanical faults.

The system is designed and demonstrated using a 5V DC motor as the machine under test. The scope of this project includes hardware design and assembly, Python-based software development, sensor data acquisition, and basic processing for motor condition monitoring. It also includes relay-based control for motor protection and simple real-time monitoring. The project does not extend to acoustic emission analysis or insulation resistance measurement.

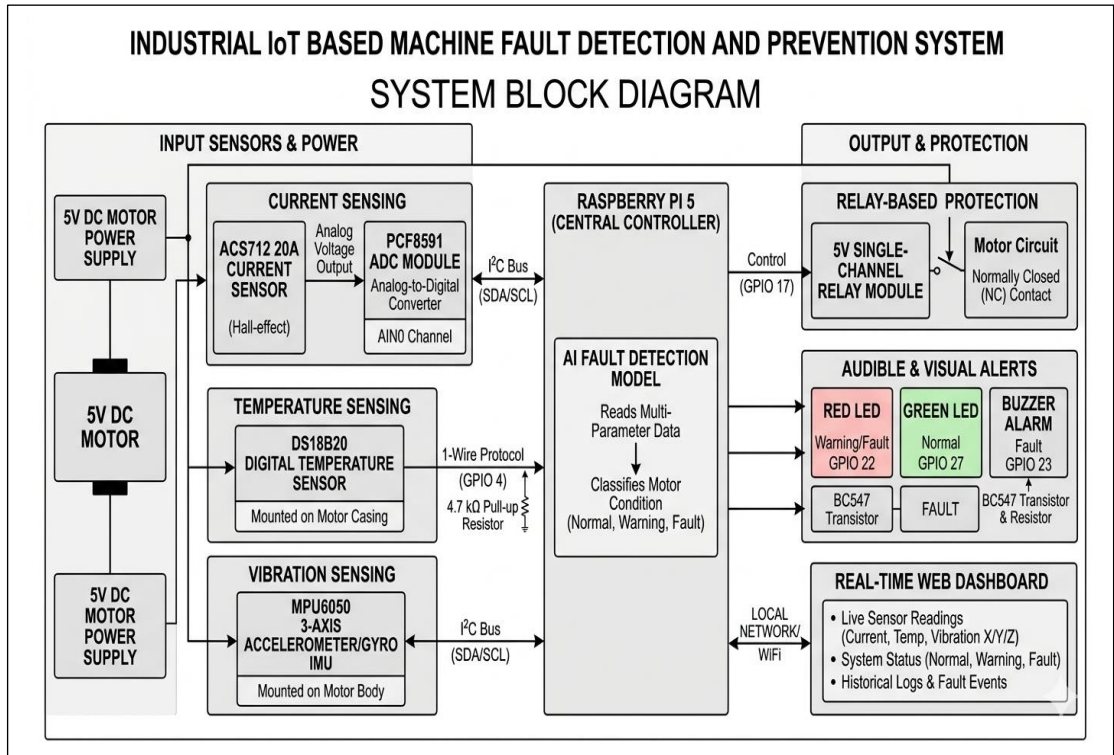
2. Circuit Diagram

2.1 System Architecture Overview

The hardware architecture of the system is organised into four functional layers: the Sensor Layer, the Signal Conditioning and Conversion Layer, the Processing Layer, and the Output and Protection Layer. Figure 2.1 illustrates the system block diagram, and Figure 2.2 presents the detailed circuit diagram with pin-level connections.

At the core of the system is the Raspberry Pi 5, which serves as the central processing unit. It communicates with the PCF8591 ADC and the MPU6050 over the shared I²C bus, and with the DS18B20 over the 1-Wire protocol on a dedicated GPIO pin. Output peripherals including LEDs, a buzzer, and a relay module are driven through GPIO output pins.

2.2 System Block Diagram



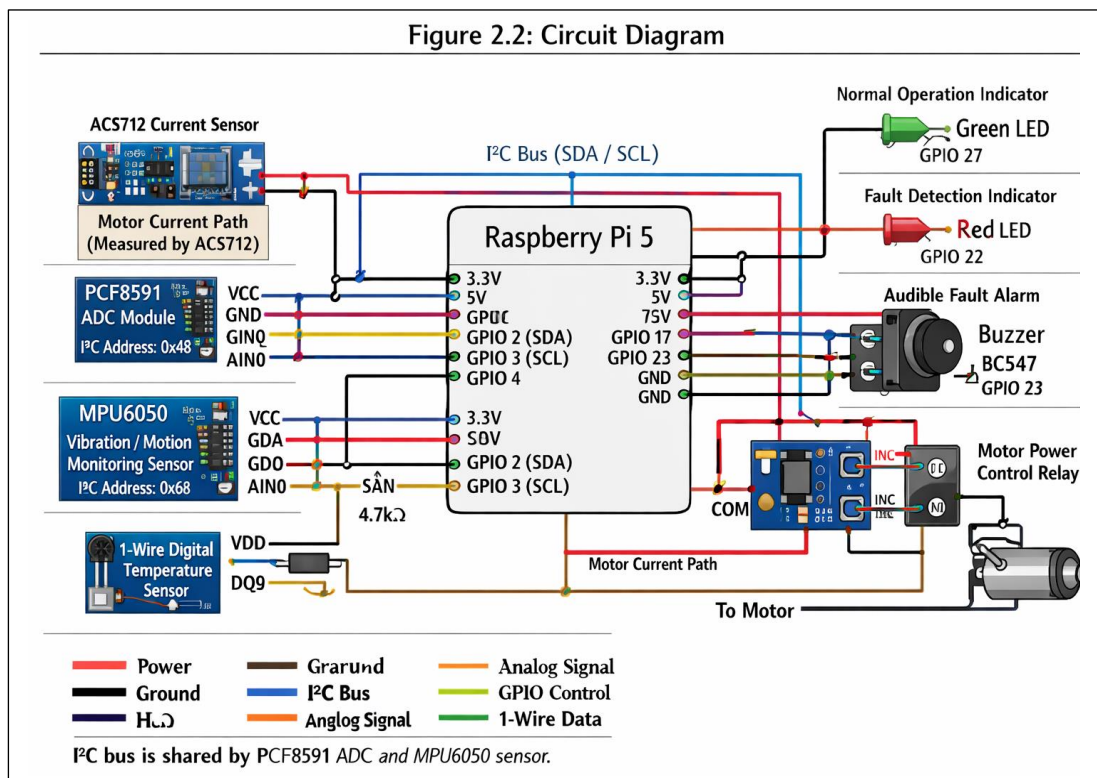
(Figure 2.1)

The signal flow through the complete system is described as follows. The 5V DC motor draws current through the ACS712 20A current sensor, which is wired in series with

the motor supply line. The analog voltage output of the ACS712 is connected to Channel AIN0 of the PCF8591 ADC. The PCF8591 digitizes this signal and transmits the result to the Raspberry Pi 5 over the I²C bus (SDA/SCL). The DS18B20 temperature sensor, mounted on the motor casing, transmits digitally encoded temperature readings directly to the Raspberry Pi 5 via the 1-Wire protocol on GPIO 4. The MPU6050 IMU, also mounted on the motor body, transmits three-axis acceleration and gyroscope data to the Raspberry Pi 5 over the I²C bus.

The Raspberry Pi 5 feeds all received sensor data into the trained AI model, which classifies the motor's condition and triggers appropriate responses: normal status is indicated by the Green LED; a warning or fault activates the Red LED and buzzer; a confirmed fault additionally drives the relay module to disconnect the motor from its power supply. Sensor data and fault events are simultaneously updated on the real-time web dashboard.

2.3 Detailed Circuit Connections



2.3.1 PCF8591 ADC Module — I²C Interface with Raspberry Pi 5

The Raspberry Pi 5 does not incorporate any native analog input pins. The PCF8591 is an 8-bit ADC with four single-ended analog input channels that communicates with the Raspberry Pi 5 over the I²C serial bus on GPIO 2 (SDA) and GPIO 3 (SCL). The I²C address is configured to 0x48 by connecting all three address pins (A0, A1, A2) to GND. The PCF8591 receives the ACS712 analog output on its AIN0 channel, digitizes it, and transmits the 8-bit result to the Raspberry Pi 5 upon request.

Table 2.1: PCF8591 ADC to Raspberry Pi 5 Pin Connections

PCF8591 Pin	Connected To	Description
VCC	3.3V — RPi 5 Pin 1	Power Supply
GND	GND — RPi 5 Pin 6	Ground Reference
SDA	GPIO 2 — RPi 5 Pin 3	I ² C Serial Data Line
SCL	GPIO 3 — RPi 5 Pin 5	I ² C Serial Clock Line
A0, A1, A2	GND	I ² C Address 0x48
AIN0	ACS712 VIOUT	Current Sensor Analog Input
AIN1–AIN3	Not Connected	Unused Channels

2.3.2 ACS712 20A Current Sensor — Connection to PCF8591

The ACS712 20A is wired in series with the motor supply line through its IP+ and IP– terminals so that all motor current flows through the sensor. The sensor outputs a quiescent voltage of 2.5V at zero current, varying linearly at 100 mV per ampere. This analog output (VIOUT) is connected to Channel AIN0 of the PCF8591 ADC for digitization.

Table 2.2: ACS712 20A Current Sensor Pin Connections

ACS712 Pin	Connected To	Description
VCC	5V — RPi 5 Pin 2	Power Supply
GND	GND — RPi 5 Pin 6	Ground Reference
IP+	Motor Power Supply Positive	Current Path Input
IP−	Motor Positive Terminal	Current Path Output
VIOUT	PCF8591 AIN0	Analog Output to ADC

2.3.3 DS18B20 Temperature Sensor — 1-Wire Interface

The DS18B20 is a digital temperature sensor that communicates using the 1-Wire protocol over a single data line connected to GPIO 4 (Pin 7) of the Raspberry Pi 5. No ADC is required as the sensor output is fully digital. A 4.7 k Ω pull-up resistor is connected between the DQ data line and the 3.3V supply rail. This resistor holds the data line in the HIGH logic state when the bus is idle, which is a mandatory requirement of the 1-Wire protocol. Without it, the data line would float to an indeterminate voltage, causing communication errors and unreliable temperature readings. The sensor is physically mounted on the motor casing for accurate surface temperature measurement.

Table 2.3: DS18B20 Temperature Sensor Pin Connections

DS18B20 Pin	Connected To	Description
VDD (+)	3.3V — RPi 5 Pin 1	Power Supply
GND (–)	GND — RPi 5 Pin 9	Ground Reference
DQ (Data)	GPIO 4 — RPi 5 Pin 7	1-Wire Data Line
DQ (Data)	VDD via 4.7 k Ω resistor	Pull-up Resistor Connection

2.3.4 MPU6050 IMU — I²C Interface with Raspberry Pi 5

The MPU6050 integrates a three-axis accelerometer and a three-axis gyroscope. It communicates directly with the Raspberry Pi 5 over the shared I²C bus at address 0x68 (AD0 pin connected to GND). Since the PCF8591 uses address 0x48 and the MPU6050 uses address 0x68, both devices coexist on the same SDA/SCL lines without conflict. The Raspberry Pi 5 reads raw acceleration data from the MPU6050 to compute vibration magnitude for fault analysis.

Table 2.4: MPU6050 IMU to Raspberry Pi 5 Pin Connections

MPU6050 Pin	Connected To	Description
VCC	3.3V — RPi 5 Pin 1	Power Supply
GND	GND — RPi 5 Pin 14	Ground Reference
SDA	GPIO 2 — RPi 5 Pin 3	I ² C Shared Data Line
SCL	GPIO 3 — RPi 5 Pin 5	I ² C Shared Clock Line
AD0	GND	I ² C Address 0x68
INT	Not Connected	Interrupt (Unused)

2.3.5 Relay Module — Automated Motor Protection

The 5V single-channel relay module is wired in series with the motor power supply through its Normally Closed (NC) contact. Under normal operation the relay contact is closed and the motor runs uninterrupted. The relay's IN pin is connected to GPIO 17 (Pin 11) of the Raspberry Pi 5. When a fault is confirmed, the Raspberry Pi 5 de-energizes the relay by changing the GPIO state, opening the NC contact and immediately disconnecting the motor from its power supply. The module includes an onboard transistor driver and flyback diode, protecting the GPIO circuitry from back-EMF.

Table 2.5: Relay Module Pin Connections

Relay Pin	Connected To	Description
VCC	5V — RPi 5 Pin 4	Relay Coil Power Supply
GND	GND — RPi 5 Pin 14	Ground Reference
IN	GPIO 17 — RPi 5 Pin 11	Fault Control Signal
COM	Motor Power Supply +ve	Common Relay Terminal
NC	Motor Positive Input	Normally Closed Contact

2.3.6 LED Indicators and Buzzer

A Green LED connected through a 330 Ω current-limiting resistor to GPIO 27 (Pin 13) remains illuminated during normal operation. A Red LED connected through a 330 Ω resistor to GPIO 22 (Pin 15) activates upon detection of a Warning or Fault condition. An active piezo buzzer is connected to GPIO 23 (Pin 16) via an NPN transistor (BC547) in a common-emitter switch configuration, with a 1 k Ω base resistor limiting gate drive current. The buzzer sounds a continuous alarm upon fault confirmation.

2.4 I²C Bus and Pull-Up Resistors

Both the PCF8591 ADC (address 0x48) and the MPU6050 IMU (address 0x68) share the same I²C bus on GPIO 2 (SDA) and GPIO 3 (SCL). Pull-up resistors of 4.7 k Ω on SDA and SCL are required to maintain correct logic HIGH levels when the bus is idle. Many breakout modules for these devices include integrated pull-up resistors; if both modules include them, external resistors are not required, as doubling the pull-up would lower the effective resistance and degrade bus signal integrity.

2.5 Power Supply

The Raspberry Pi 5 is powered via its USB-C input using a 5V, 5A adapter. The 3.3V GPIO rail powers the PCF8591, MPU6050, and DS18B20. The ACS712 and relay module are powered from the 5V GPIO rail. The 5V DC motor is powered from a dedicated, isolated external supply to prevent motor switching transients from affecting the measurement circuitry.

3. Component List

Table 3.1 provides a complete list of all hardware components required for the construction of the system.

Table 3.1: Complete Component List

S. No.	Component	Specification / Model	Qty.
1	Raspberry Pi 4 Model B (4GB)	Raspberry Pi 4 Model B (4GB RAM)	1
2	Current Sensor	ACS712 — 20A Range	1
3	Temperature Sensor	DS18B20 Probe	1
4	Vibration Sensor	MPU6050 (3-axis Accel + 3-axis Gyro)	1
5	ADC Module	PCF8591, 8-bit, 4-ch, I ² C	1
6	Relay Module	5V Single-Channel Relay Module	1
7	DC Motor (Device Under Test)	5V DC Motor	1
8	Pull-up Resistor	4.7 k Ω , 1/4W	1
9	LED — Green	5mm, Green, 20mA max	1
10	LED — Red	5mm, Red, 20mA max	1
11	MicroSD Card (32GB, Class 10)	SanDisk Ultra 32GB	1
12	Active Piezo Buzzer	5V Active Buzzer	1
13	Power Adapter	5V/ 5A USB-C	1

S. No.	Component	Specification / Model	Qty.
14	Motor Power Supply	5V / 1A DC Adapter	1
15	Breadboard	830-tie Solderless Breadboard	1
16	Jumper Wires	Male-to-Male and Male-to-Female	1

4. Component Description

4.1 Raspberry Pi 4

The Raspberry Pi 4 Model B serves as the central processing unit of the proposed Industrial IoT based machine fault detection and prevention system. It is a compact single-board computer equipped with a Broadcom BCM2711 quad-core Cortex-A72 processor operating at 1.5 GHz and 4GB of RAM. The board provides multiple General Purpose Input/Output (GPIO) pins which allow interfacing with external sensors, actuators, and communication modules.

In this project, the Raspberry Pi collects sensor data from the ACS712 current sensor (through the PCF8591 ADC module), the DS18B20 temperature sensor via the 1-Wire protocol, and the MPU6050 vibration sensor through the I²C interface. The Raspberry Pi processes the incoming data to detect abnormal machine conditions such as overcurrent, excessive temperature, or abnormal vibration. Based on the analysis, it controls the relay module to stop the motor and activates warning indicators such as LEDs and a buzzer. Additionally, the Raspberry Pi enables IoT functionality by sending real-time monitoring data to a remote dashboard via its built-in Wi-Fi connectivity.

4.2 ACS712 20A Current Sensor

The ACS712 is a Hall-effect based linear current sensor used to measure the current flowing through an electrical conductor. It can measure both AC and DC currents and provides an analog voltage output proportional to the measured current. The module used in this project has a measurement range of $\pm 20\text{A}$.

In this system, the ACS712 sensor is connected in series with the motor power supply so that the motor current passes through the sensor. The output voltage of the sensor varies according to the current flowing through it. This analog signal is then converted into digital data using the PCF8591 Analog-to-Digital Converter. By monitoring the current drawn by the motor, the system can detect abnormal conditions such as overload, short circuits, or mechanical faults.

4.3 DS18B20 Digital Temperature Sensor

The DS18B20 is a digital temperature sensor that communicates using the 1-Wire protocol. It provides accurate temperature measurements with a typical accuracy of $\pm 0.5^{\circ}\text{C}$ over a wide temperature range. The sensor requires only one data line for communication with the microcontroller along with power and ground connections.

In this project, the DS18B20 temperature probe is used to monitor the temperature of the motor or surrounding machine components. A $4.7\text{ k}\Omega$ pull-up resistor is connected between the data line and the supply voltage to ensure proper communication on the 1-Wire bus. The Raspberry Pi reads temperature values from the sensor through its GPIO pin and continuously checks for abnormal temperature rise that may indicate overheating or potential failure.

4.4 MPU6050 Inertial Measurement Unit

The MPU6050 is a 6-axis motion tracking sensor that combines a 3-axis accelerometer and a 3-axis gyroscope in a single module. It communicates with microcontrollers using the I²C interface and is widely used for motion detection, vibration monitoring, and orientation tracking.

In this system, the MPU6050 is used to monitor machine vibrations and motion characteristics. Abnormal vibration patterns can indicate faults such as imbalance, misalignment, or bearing wear in rotating machinery. The sensor continuously sends acceleration and angular velocity data to the Raspberry Pi through the shared I²C bus. The Raspberry Pi analyzes these readings to detect unusual vibration levels and trigger alerts if necessary.

4.5 PCF8591 ADC Module

The PCF8591 is an 8-bit Analog-to-Digital Converter (ADC) that provides four analog input channels and one analog output channel. It communicates with microcontrollers using the I²C communication protocol.

Since the Raspberry Pi does not have built-in analog input capability, the PCF8591 module is used to convert analog signals into digital data. In this project, the analog output from the ACS712 current sensor is connected to one of the PCF8591 input channels. The ADC converts the analog voltage into a digital value that can be processed by the Raspberry Pi.

3.6 Relay Module (5V Single Channel)

The relay module is an electrically operated switch used to control high-power devices using low-power signals from a microcontroller. The module contains an electromagnetic relay along with a transistor driver circuit for easy interfacing.

In this project, the relay module is used to control the power supply to the DC motor. The Raspberry Pi sends a control signal to the relay module through one of its GPIO pins. When a fault condition such as excessive current, temperature, or vibration is detected, the Raspberry Pi activates the relay to disconnect the motor power supply, thereby preventing potential damage.

4.7 5V DC Motor

The 5V DC motor acts as the device under test in this project. It represents the industrial machine whose operating parameters are continuously monitored. The motor draws current from an external power supply and performs mechanical rotation when powered.

The current drawn by the motor is measured using the ACS712 current sensor, while temperature and vibration around the motor are monitored using the DS18B20 and

MPU6050 sensors. If abnormal conditions are detected, the system disconnects the motor through the relay module to prevent further damage.

4.8 Supporting Components

Several supporting components are used in the system to ensure proper operation, indication, and power management of the Industrial IoT based machine fault detection and prevention system.

A **4.7 k Ω pull-up resistor** is used in the DS18B20 temperature sensing circuit. Since the DS18B20 operates using the 1-Wire communication protocol, the data line must be pulled high to maintain a stable logic level when the line is idle. The resistor is connected between the data line and the 3.3V supply to ensure reliable communication between the sensor and the Raspberry Pi.

Two **LED indicators (Green and Red)** are used to provide visual feedback about the system status. The green LED indicates normal operating conditions of the machine, while the red LED indicates that a fault condition has been detected. Each LED is connected to a Raspberry Pi GPIO pin through a current-limiting resistor to protect the LED from excessive current.

An **active piezo buzzer** is used to generate an audible alarm when abnormal operating conditions are detected. The buzzer is controlled through a transistor driver circuit connected to a Raspberry Pi GPIO pin. When the system detects excessive current, high temperature, or abnormal vibration, the buzzer is activated to alert the operator immediately.

A **MicroSD card (32GB)** is used as the primary storage device for the Raspberry Pi. It stores the operating system, program files, and sensor monitoring scripts required for the operation of the system. It may also store logged sensor data for monitoring and analysis.

Finally, a **5V / 5A USB-C power adapter** is used to supply stable power to the Raspberry Pi and connected peripherals. Providing a reliable power source is essential to ensure continuous and stable operation of the monitoring system.

5. Costing of the Project

Table 5.1 presents the cost estimate for all components required to build the prototype system. Prices are indicative and based on current market rates; actual costs may vary depending on the supplier and region.

Table 5.1: Project Component Cost Breakdown

S. No.	Component	Specification / Model	Unit Cost (₹)
1	Raspberry Pi 4 Model B (4GB)	Raspberry Pi 4 Model B (4GB RAM)	6,500
2	Current Sensor	ACS712 — 20A Range	180
3	Temperature Sensor	DS18B20 Probe	80
4	Vibration Sensor	MPU6050 (3-axis Accel + 3-axis Gyro)	150
5	ADC Module	PCF8591, 8-bit, 4-ch, I ² C	120
6	Relay Module	5V Single-Channel Relay Module	80
7	DC Motor (Device Under Test)	5V DC Motor	200
8	Pull-up Resistor	4.7 k Ω , 1/4W	2
9	LED — Green	5mm, Green, 20mA max	5
10	LED — Red	5mm, Red, 20mA max	5
11	MicroSD Card (32GB, Class 10)	SanDisk Ultra 32GB	350
12	Active Piezo Buzzer	5V Active Buzzer	30
13	Power Adapter	5V/ 5A USB-C	600

S. No.	Component	Specification / Model	Unit Cost (₹)
14	Motor Power Supply	5V / 1A DC Adapter	250
15	Breadboard	830-tie Solderless Breadboard	100
16	Jumper Wires(Assorted Pack)	Male-to-Male and Male-to-Female	100

6. Advantages and Disadvantages

6.1 Advantages

- **Multi-Parameter Fault Detection:** By simultaneously monitoring current, temperature, and vibration, the system detects a significantly broader range of fault types — including both electrical and mechanical failures — compared to systems that monitor only a single parameter.
- **AI-Based Classification:** The use of a trained AI model for fault classification provides greater accuracy and nuance than simple threshold comparisons, enabling the system to distinguish between normal transient conditions (such as motor startup inrush) and genuine fault conditions.
- **Automated Protective Action:** The relay-based motor disconnection mechanism responds to confirmed faults without requiring human intervention, providing immediate hardware-level protection and minimizing the risk of further machine damage.
- **Real-Time Web Dashboard:** The integrated web-based monitoring dashboard provides operators with a centralized, accessible interface for live sensor data, system status, and fault history, viewable from any networked device without requiring specialized software.
- **Early Fault Detection:** Continuous monitoring allows the system to identify deteriorating conditions at an early stage and significantly reducing repair costs and production downtime.
- **Cost-Effective Design:** The use of commercially available, low-cost sensor modules and the Raspberry Pi 5 results in a highly economical monitoring system compared to proprietary industrial condition monitoring equipment.
- **Scalable Architecture:** The modular design allows additional sensors or monitored machines to be added with minimal changes to the existing hardware and software architecture.
- **Galvanic Isolation in Current Sensing:** The ACS712 provides electrical isolation between the motor power circuit and the measurement system, enhancing operator safety and signal integrity.

-
- **Data Logging and Trend Analysis:** All sensor readings and fault events are logged with timestamps, enabling post-event analysis and the identification of degradation trends for predictive maintenance planning.

6.2 Disadvantages

- **Raspberry Pi 5 Not Rated for Harsh Industrial Environments:** The Raspberry Pi 5 is an educational and prototyping platform not specifically designed for industrial conditions involving wide temperature ranges, high humidity, dust, vibration, or strong electromagnetic interference.
- **Limited ADC Resolution:** The PCF8591 provides only 8-bit resolution (256 discrete levels), which may limit the precision of current measurements in applications requiring finer granularity. A higher-resolution ADC such as the ADS1115 (16-bit) would improve measurement accuracy.
- **AI Model Dependency on Training Data:** The accuracy of the AI fault classification model is directly dependent on the quality and diversity of the training dataset. A model trained on limited data may not generalize well to fault conditions not represented during training.
- **Single Motor Monitoring in Current Prototype:** The current prototype is designed to monitor a single 5V DC motor. Scaling the system to simultaneously monitor multiple industrial machines would require additional hardware nodes and a more complex software architecture.
- **Internet Dependency for Remote Dashboard Access:** Remote access to the web dashboard requires stable network connectivity.
- **Raspberry Pi 5 Not a Real-Time System:** The Raspberry Pi 5 runs a general-purpose Linux-based OS, which is not a Real-Time Operating System (RTOS).
- **In scenarios requiring extremely precise sampling intervals,** the OS scheduler may introduce timing jitter.
- **DS18B20 Conversion Latency:** At 12-bit resolution, the DS18B20 requires up to 750 milliseconds for a single temperature conversion, which limits the maximum temperature sampling rate to approximately 1.3 readings per second. This is generally adequate for thermal monitoring but may be insufficient for detecting rapid temperature transients.

7. Conclusion

This project demonstrates the design and implementation of an Industrial IoT-based motor monitoring and protection system capable of real-time, multi-parameter observation of a 5V DC motor. The system integrates sensors such as the ACS712 20A Current Sensor, DS18B20 Temperature Sensor, and MPU6050 to monitor both electrical and mechanical aspects of motor operation.

The PCF8591 ADC Module is used to interface the analog current sensor with the Raspberry Pi 5, while the temperature and vibration sensors communicate digitally using standard protocols, ensuring a simple and efficient system design.

The collected sensor data is processed using a basic classification approach to determine the motor's condition as either Normal or Jam. Upon detection of a Jam condition, the system responds automatically by activating LED and buzzer alerts and disconnecting the motor using a relay module to prevent potential damage.

The system also provides basic real-time monitoring through an IoT interface, allowing users to observe sensor readings and system status remotely.

Overall, this project highlights the practical implementation of multi-parameter sensing and embedded system-based monitoring for motor protection. It demonstrates that an effective and responsive monitoring system can be developed using low-cost and easily available components, making it suitable for small-scale industrial or educational applications. The project successfully meets its intended objectives and provides a foundation for future enhancements.

8. Future Scope

The current implementation provides a robust platform for numerous enhancements that could extend the system's capability, accuracy, and applicability in industrial environments:

- **Higher-Resolution ADC:** Replacing the PCF8591 (8-bit) with a higher-resolution ADC such as the ADS1115 (16-bit) would significantly improve the precision of current measurements, enabling the detection of more subtle current variations associated with early-stage electrical faults.
- **Advanced AI Models:** Upgrading from a basic classifier to more sophisticated machine learning architectures such as Long Short-Term Memory (LSTM) networks or Convolutional Neural Networks (CNNs) trained on time-series sensor data would enable the detection of nuanced fault patterns and improve classification accuracy across a wider range of fault types.
- **Voltage Monitoring:** Adding a voltage sensing module to the system would enable the calculation of real electrical power, power factor, and efficiency, providing a more comprehensive picture of the motor's electrical health.
- **Acoustic Emission Sensing:** Integrating a microphone or dedicated acoustic emission sensor to capture high-frequency sound produced by bearing wear or arcing would add another dimension to fault detection that complements the existing vibration and electrical measurements.
- **Industrial Communication Protocols:** Implementing Modbus TCP/RTU or OPC-UA communication would allow the system to integrate with existing industrial SCADA, DCS, and MES platforms, enabling deployment within established industrial automation infrastructure.
- **Multi-Machine Network:** Deploying multiple sensor nodes, each monitoring a different machine and connected to a central server via MQTT or LoRaWAN, could create a plant-wide machine health monitoring network with a unified dashboard.
- **Edge AI Acceleration:** Deploying lightweight TensorFlow Lite or ONNX models on an edge AI accelerator such as the Google Coral USB Accelerator would enable faster on-device AI inference with reduced CPU load on the Raspberry Pi 5.

-
- **Mobile Application:** Developing a dedicated Android or iOS application for real-time alert notifications, sensor data visualization, and remote relay control would improve operator accessibility and responsiveness.
 - **Cloud Analytics Integration:** Integrating with cloud IoT platforms such as AWS IoT Core, Microsoft Azure IoT Hub, or Google Cloud IoT would enable advanced analytics, automated maintenance scheduling, and large-scale data storage for long-term trend analysis.
 - **Digital Twin:** Creating a digital model of the monitored motor synchronized with real-time sensor data would enable simulation of fault propagation scenarios and support optimization of maintenance strategies without requiring physical intervention on the machine.

Appendix A — Python Program Code

A.1 Main Fault Detection Script (realtime.py)

The following Python script is the main program executed on the Raspberry Pi 5. It initializes all sensor interfaces, reads data from the ACS712 via the PCF8591 ADC over I²C, from the DS18B20 via 1-Wire, and from the MPU6050 via I²C. All sensor readings are fed into the trained AI model, which returns a fault classification. The script then drives GPIO outputs for LEDs, buzzer, and relay accordingly, logs all events to a CSV file, and updates the dashboard.

```
import smbus2
import sys
sys.modules['smbus'] = smbus2
import time
import numpy as np
import pandas as pd
import joblib
import smbus2 as smbus
from w1thermsensor import W1ThermSensor
from mpu6050 import mpu6050
import RPi.GPIO as GPIO

# -----
# LOAD MODEL FILES
# -----
model = joblib.load("rf_model.pkl")
scaler = joblib.load("scaler.pkl")
feature_columns = joblib.load("features.pkl")
WINDOW_SIZE = joblib.load("window.pkl")

# -----
# SETUP I2C (PCF8591)
# -----
bus = smbus.SMBus(1)
PCF8591_ADDR = 0x48
```

```
def read_adc(channel):
    bus.write_byte(PCF8591_ADDR, channel)
    bus.read_byte(PCF8591_ADDR)
    return bus.read_byte(PCF8591_ADDR)

# -----
# CURRENT SENSOR (ACS712)
# -----
def read_current():
    value = read_adc(0)
    voltage = value * (3.3 / 255)
    current = (voltage - 2.5) / 0.185
    return current

# -----
# TEMPERATURE (DS18B20)
# -----
temp_sensor = W1ThermSensor()

def read_temp():
    return temp_sensor.get_temperature()

# -----
# VIBRATION (MPU6050)
# -----
mpu = mpu6050(0x68)

def read_vibration():
    data = mpu.get_accel_data()
    vib = abs(data['x']) + abs(data['y']) + abs(data['z'])
    return vib

# -----
import smbus2
```

```
import sys
sys.modules['smbus'] = smbus2
import time
import numpy as np
import pandas as pd
import joblib
import smbus2 as smbus
from w1thermsensor import W1ThermSensor
from mpu6050 import mpu6050
import RPi.GPIO as GPIO

# -----
# LOAD MODEL FILES
# -----
model = joblib.load("rf_model.pkl")
scaler = joblib.load("scaler.pkl")
feature_columns = joblib.load("features.pkl")
WINDOW_SIZE = joblib.load("window.pkl")

# -----
# SETUP I2C (PCF8591)
# -----
bus = smbus.SMBus(1)
PCF8591_ADDR = 0x48

def read_adc(channel):
    bus.write_byte(PCF8591_ADDR, channel)
    bus.read_byte(PCF8591_ADDR)
    return bus.read_byte(PCF8591_ADDR)

# -----
# CURRENT SENSOR (ACS712)
# -----
def read_current():
```

```
    value = read_adc(0)
    voltage = value * (3.3 / 255)
    current = (voltage - 2.5) / 0.185
    return current

# -----
# TEMPERATURE (DS18B20)
# -----
temp_sensor = W1ThermSensor()

def read_temp():
    return temp_sensor.get_temperature()

# -----
# VIBRATION (MPU6050)
# -----
mpu = mpu6050(0x68)

def read_vibration():
    data = mpu.get_accel_data()
    vib = abs(data['x']) + abs(data['y']) + abs(data['z'])
    return vib

# -----
# RELAY SETUP
# -----
GPIO.setmode(GPIO.BCM)
GPIO.setup(22, GPIO.OUT)

def motor_on():
    GPIO.output(22, 0)

def motor_off():
    GPIO.output(22, 1)
```

```

# -----
# LED + BUZZER SETUP
# -----

GREEN_LED = 17
RED_LED = 27
BUZZER = 23

GPIO.setup(GREEN_LED, GPIO.OUT)
GPIO.setup(RED_LED, GPIO.OUT)
GPIO.setup(BUZZER, GPIO.OUT)
def normal_state():
    GPIO.output(GREEN_LED, 1) # Green ON
    GPIO.output(RED_LED, 0)   # Red OFF
    GPIO.output(BUZZER, 1)    # Buzzer OFF

def jam_state():
    GPIO.output(GREEN_LED, 0) # Green OFF
    GPIO.output(RED_LED, 1)   # Red ON
    GPIO.output(BUZZER, 0)    # Buzzer ON

# -----
# BUFFERS
# -----

curr_buffer = []
temp_buffer = []
vib_buffer = []

# -----
# FEATURE EXTRACTION
# -----

def extract_features(curr, temp, vib):

    curr = np.array(curr)
    temp = np.array(temp)
    vib = np.array(vib)

```

```

return {
    "mean_curr": curr.mean(),
    "rms_curr": np.sqrt(np.mean(curr**2)),
    "std_curr": curr.std(),
    "mean_temp": temp.mean(),
    "temp_rate": (temp[-1] - temp[0]) / len(temp),
    "rms_vib": np.sqrt(np.mean(vib**2)),
    "std_vib": vib.std(),
    "peak_vib": vib.max(),
    "crest": vib.max() / (np.sqrt(np.mean(vib**2)) + 1e-6),
    "kurtosis": np.mean((vib - vib.mean())**4) / (vib.std()**4 + 1e-6)
}

# -----
# MAIN LOOP
# -----

print("System started...")

while True:

    curr = read_current()
    temp = read_temp()
    vib = read_vibration()

    print(f"Curr:{curr:.2f} Temp:{temp:.2f} Vib:{vib:.2f}")

    curr_buffer.append(curr)
    temp_buffer.append(temp)
    vib_buffer.append(vib)

    if len(curr_buffer) >= WINDOW_SIZE:

        features_dict = extract_features(curr_buffer, temp_buffer, vib_buffer)

```

```

features_df = pd.DataFrame([features_dict])
features_df = features_df[feature_columns]
features_scaled = scaler.transform(features_df)

probs = model.predict_proba(features_scaled)
ml_pred = model.predict(features_scaled)[0]

# -----
# SIMPLE THRESHOLD LOGIC
# -----

curr_val = features_df["mean_curr"].values[0]
peak_vib = features_df["peak_vib"].values[0]

# ---- SIMPLE 2-CLASS LOGIC ----
if curr_val >= 0.55:
    prediction = "jam"
else:
    prediction = "normal"
if prediction == "jam":
    print("⚠ JAM detected! Turning motor OFF")
    GPIO.output(22, GPIO.HIGH)
    jam_state()
else:
    print("Motor running → ON")
    GPIO.output(22, GPIO.LOW)
    normal_state()
print("GPIO 22 value:", GPIO.input(22))
print("Relay set to:", "OFF" if prediction=="jam" else "ON")
print("GPIO state:", GPIO.input(22))
print(f"[STATE]          {prediction.upper()}          |          Curr:{curr_val:.2f}
PeakVib:{peak_vib:.2f}")
print("Final Prediction:", prediction)
print("Probabilities:", probs)

```

```
print("Classes:", model.classes_)

# MOTOR CONTROL
if prediction != "normal":
    print("Fault detected! Turning motor OFF")
    motor_off()
else:
    motor_on()
log = pd.DataFrame([{
    "Current": curr,
    "Temp": temp,
    "Vibration": vib,
    "State": prediction
}])

log.to_csv("data_log.csv", mode='a',
          header=not pd.io.common.file_exists("data_log.csv"),
          index=False)

# RESET BUFFERS
curr_buffer.clear()
temp_buffer.clear()
vib_buffer.clear()

time.sleep(0.2)
```

A.2 Model Training Script (motor_fault_project.py)

The following script demonstrates how a classification approach is implemented using labelled sensor data collected from the motor under Normal and Jam conditions. The processed data or model parameters are saved and later used by the main script during execution.

```
import pandas as pd
import numpy as np
```

```
import matplotlib.pyplot as plt
import joblib

from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier
from sklearn.metrics import accuracy_score, confusion_matrix,
ConfusionMatrixDisplay, classification_report
from sklearn.preprocessing import StandardScaler

# -----
# 1. Load dataset
# -----
df = pd.read_csv("final_dataset.csv")

# -----
# 2. Split features & label
# -----
X = df.drop("label", axis=1)
y = df["label"]

# -----
# 3. Train-test split
# -----
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# -----
# 4. Feature scaling
# -----
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
```

```
X_test = scaler.transform(X_test)

# -----
# 5. Models
# -----
rf_model = RandomForestClassifier(n_estimators=100, random_state=42)
gb_model = GradientBoostingClassifier(n_estimators=100, learning_rate=0.1)

# -----
# 6. Train
# -----
rf_model.fit(X_train, y_train)
gb_model.fit(X_train, y_train)

# -----
# 7. Predictions
# -----
rf_pred = rf_model.predict(X_test)
gb_pred = gb_model.predict(X_test)

# -----
# 8. Accuracy
# -----
rf_acc = accuracy_score(y_test, rf_pred)
gb_acc = accuracy_score(y_test, gb_pred)

print("Random Forest Accuracy:", rf_acc)
print("Gradient Boost Accuracy:", gb_acc)

# -----
# 9. Classification report
# -----
print("\nRandom Forest Report:\n", classification_report(y_test, rf_pred))
print("\nGradient Boost Report:\n", classification_report(y_test, gb_pred))
```

```
# -----
# 10. Confusion Matrix
# -----

cm_rf = confusion_matrix(y_test, rf_pred)
cm_gb = confusion_matrix(y_test, gb_pred)

ConfusionMatrixDisplay(cm_rf).plot()
plt.title("Random Forest Confusion Matrix")
plt.show()

ConfusionMatrixDisplay(cm_gb).plot()
plt.title("Gradient Boost Confusion Matrix")
plt.show()

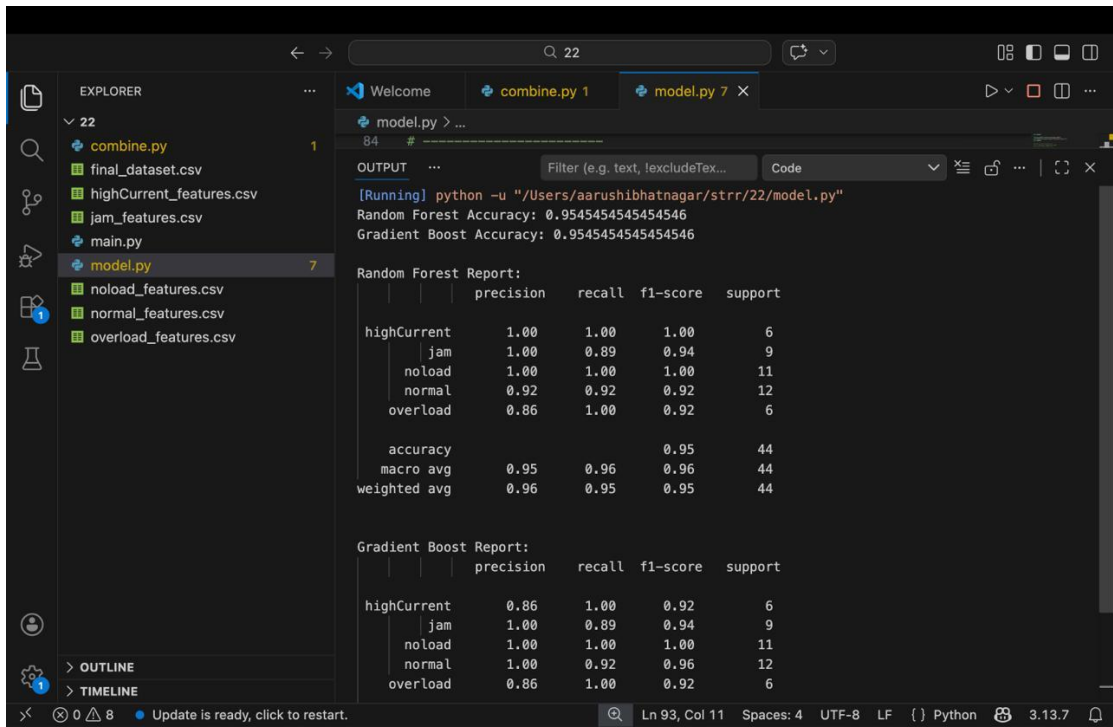
# -----
# 11. Accuracy comparison plot
# -----

models = ["Random Forest", "Gradient Boost"]
accuracies = [rf_acc, gb_acc]

plt.figure()
plt.bar(models, accuracies)
plt.xlabel("Model")
plt.ylabel("Accuracy")
plt.title("Model Comparison")
plt.show()

joblib.dump(rf_model, "rf_model.pkl")
joblib.dump(scaler, "scaler.pkl")
```

Accuracy and Prediction Result



The screenshot shows the Visual Studio Code interface with a file explorer on the left and a code editor on the right. The file explorer shows a project named '22' with files: combine.py, final_dataset.csv, highCurrent_features.csv, jam_features.csv, main.py, model.py, noload_features.csv, normal_features.csv, and overload_features.csv. The code editor shows the output of a Python script running 'model.py'. The output displays the Random Forest Accuracy: 0.9545454545454546 and Gradient Boost Accuracy: 0.9545454545454546. Below the accuracy reports, there are two tables: Random Forest Report and Gradient Boost Report, both showing precision, recall, f1-score, and support for different classes.

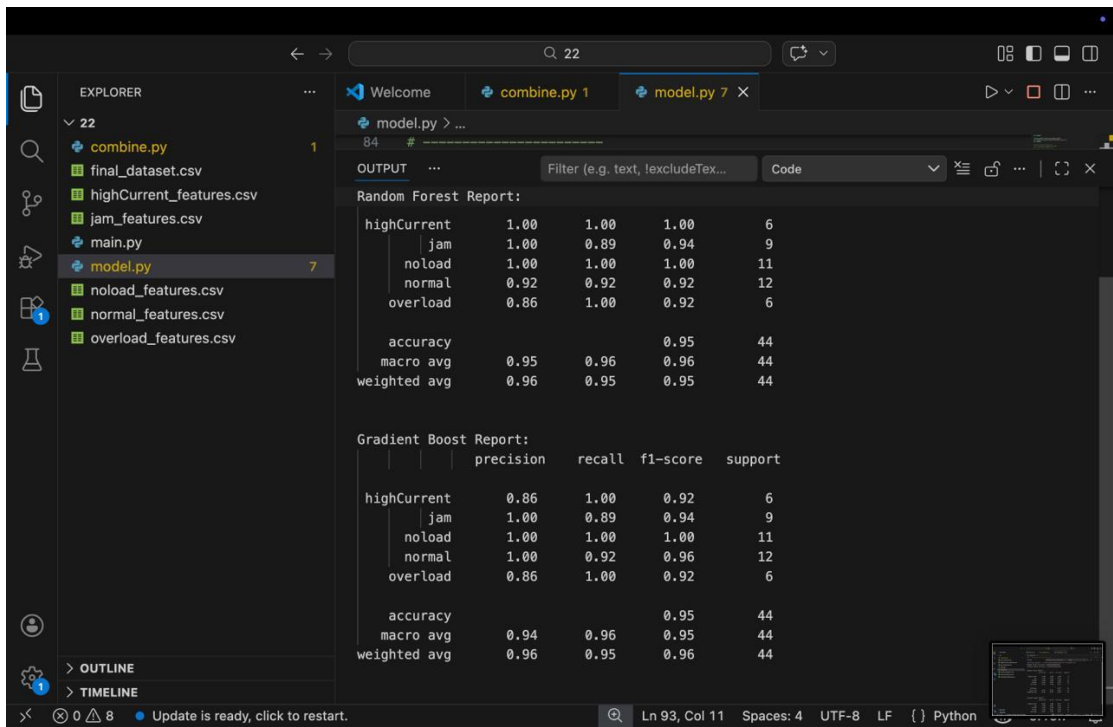
```
[Running] python -u "/Users/aarushibhatnagar/strr/22/model.py"
Random Forest Accuracy: 0.9545454545454546
Gradient Boost Accuracy: 0.9545454545454546
```

Random Forest Report:

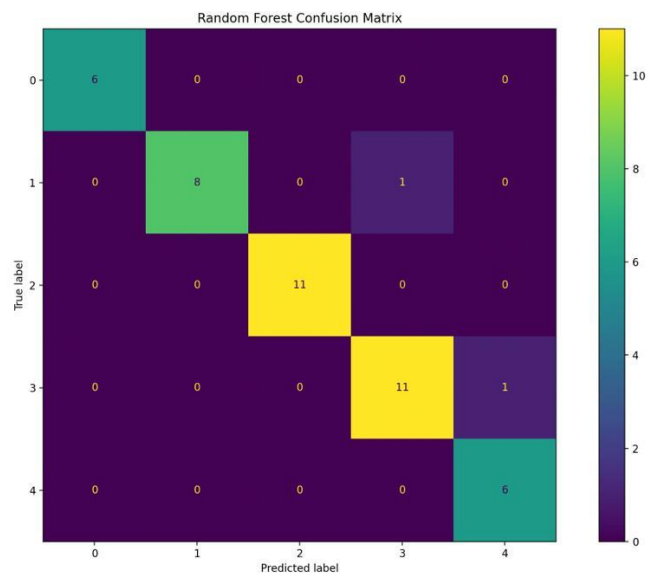
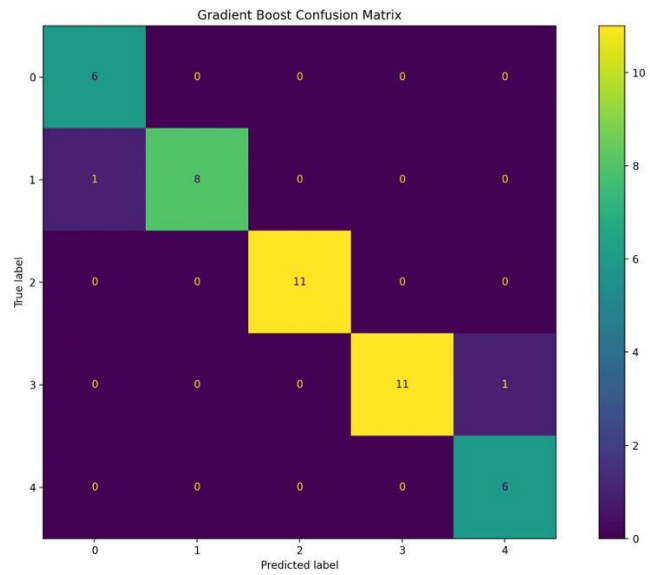
	precision	recall	f1-score	support
highCurrent	1.00	1.00	1.00	6
jam	1.00	0.89	0.94	9
noload	1.00	1.00	1.00	11
normal	0.92	0.92	0.92	12
overload	0.86	1.00	0.92	6
accuracy			0.95	44
macro avg	0.95	0.96	0.96	44
weighted avg	0.96	0.95	0.95	44

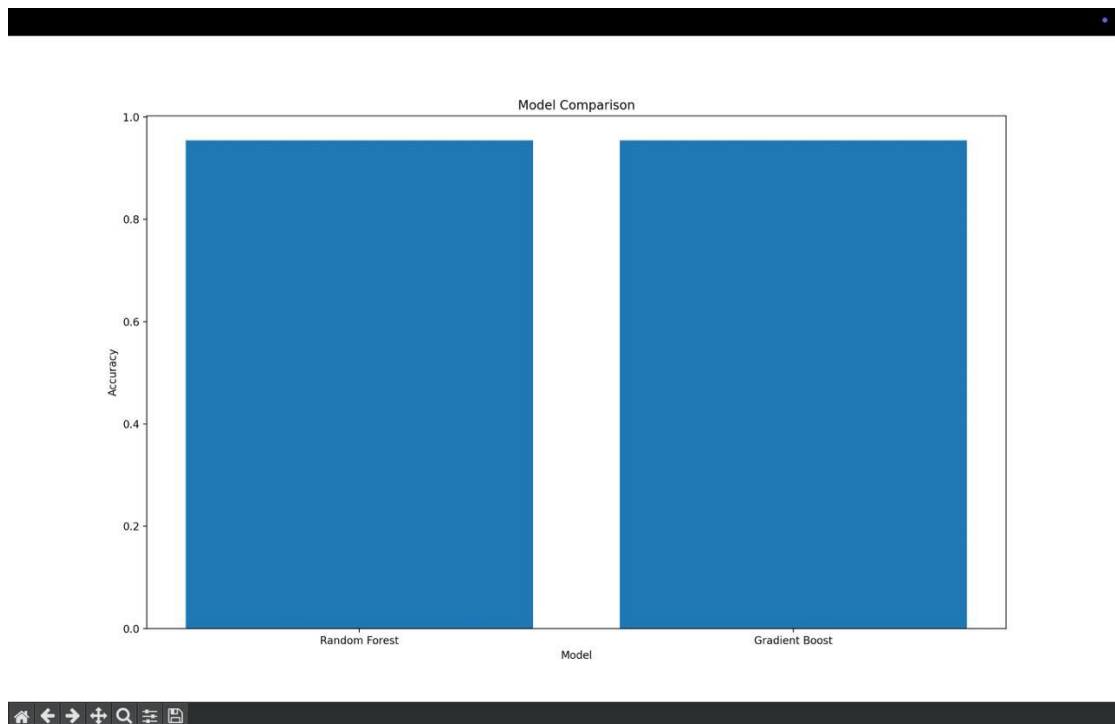
Gradient Boost Report:

	precision	recall	f1-score	support
highCurrent	0.86	1.00	0.92	6
jam	1.00	0.89	0.94	9
noload	1.00	1.00	1.00	11
normal	1.00	0.92	0.96	12
overload	0.86	1.00	0.92	6
accuracy			0.95	44
macro avg	0.94	0.96	0.96	44
weighted avg	0.96	0.95	0.96	44



This screenshot is identical to the one above, showing the same VS Code interface and output results for the Random Forest and Gradient Boost models.





A.3 Relay Script (relay.py)

```
import RPi.GPIO as GPIO
import time

GPIO.setmode(GPIO.BCM)
GPIO.setup(22, GPIO.OUT)

while True:
    GPIO.output(22, 0) # ON (click)
    print("Relay ON")
    time.sleep(2)

    GPIO.output(22, 1) # OFF
    print("Relay OFF")
    time.sleep(2)
```

A4. Dashboard Script

```
import streamlit as st
import pandas as pd
import time
import os

st.set_page_config(page_title="Motor Monitoring", layout="wide")

CSV_FILE = "data_log.csv"

st.title("⚙ Motor Fault Detection Dashboard")

# placeholders
metric_placeholder = st.empty()
chart_placeholder = st.empty()
table_placeholder = st.empty()

def load_data():
    if not os.path.exists(CSV_FILE):
        return None
    try:
        df = pd.read_csv(CSV_FILE)
        return df
    except:
        return None

while True:
    df = load_data()

    if df is None or df.empty:
        st.warning("Waiting for data...")
        time.sleep(1)
        st.rerun()
```

```
latest = df.iloc[-1]

current = latest["Current"]
temp = latest["Temp"]
vib = latest["Vibration"]
state = str(latest["State"]).lower()

# -----
# METRICS
# -----

with metric_placeholder.container():
    col1, col2, col3, col4 = st.columns(4)

    col1.metric("Current (A)", f"{current:.2f}")
    col2.metric("Temperature (°C)", f"{temp:.2f}")
    col3.metric("Vibration", f"{vib:.2f}")

    if state == "jam":
        col4.metric("State", "JAM 🚫")
        st.error("⚠️ JAM DETECTED - Motor OFF")
    else:
        col4.metric("State", "NORMAL ✅")

# -----
# CHARTS
# -----

with chart_placeholder.container():
    st.subheader("Sensor Graphs")

    df_small = df.tail(50)

    st.line_chart(df_small[["Current"]], height=200)
```

```
st.line_chart(df_small[["Vibration"]], height=200)
```

```
st.line_chart(df_small[["Temp"]], height=200)
```

```
# -----
```

```
# TABLE
```

```
# -----
```

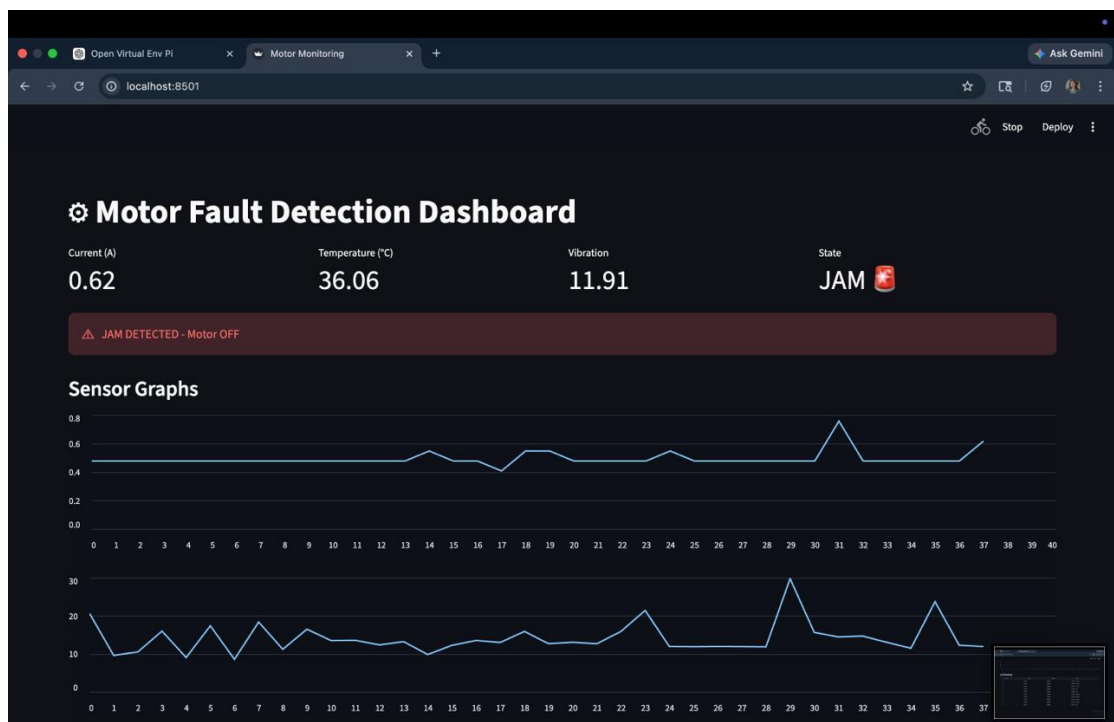
```
with table_placeholder.container():
```

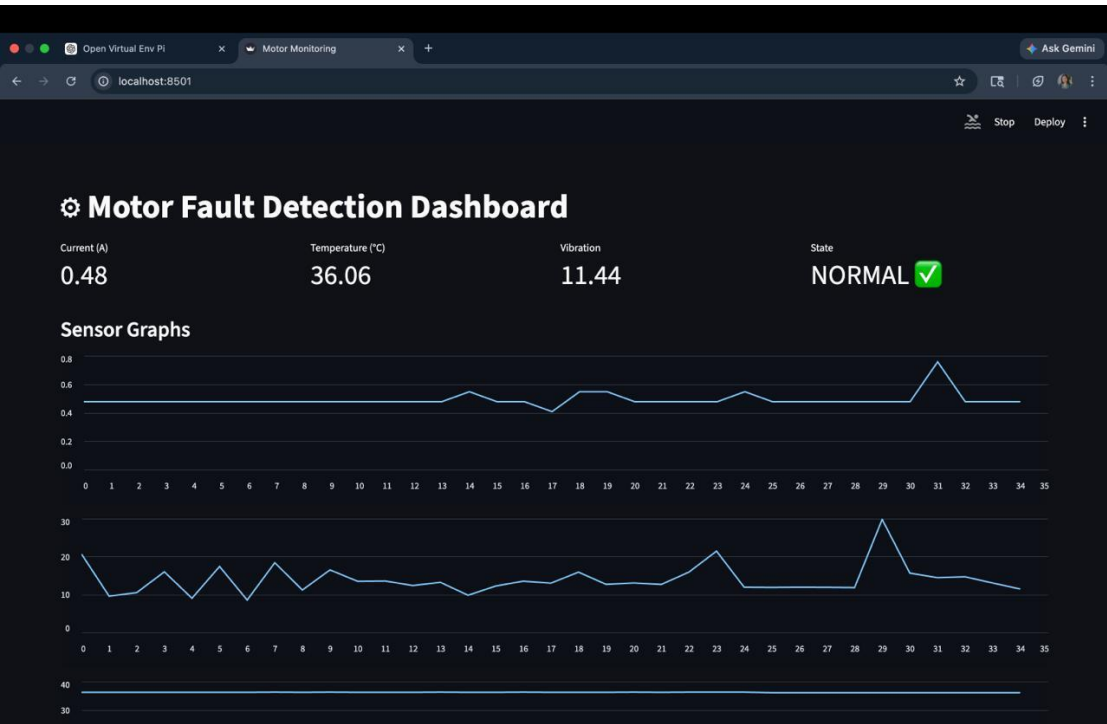
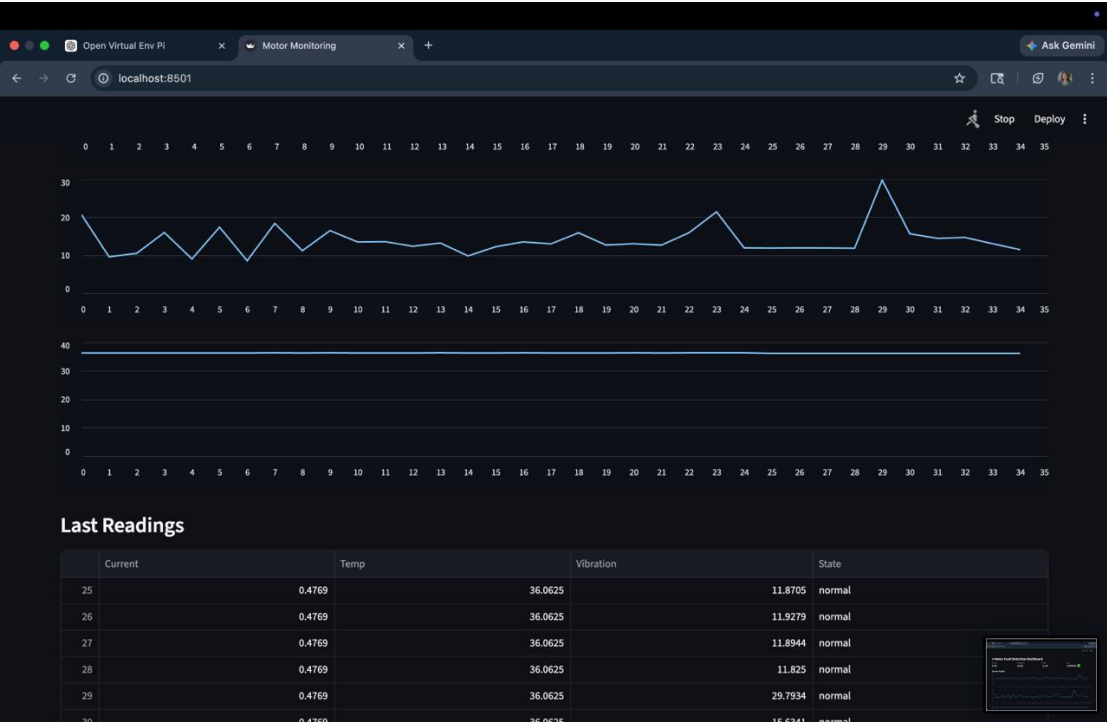
```
    st.subheader("Last Readings")
```

```
    st.dataframe(df.tail(10), width="stretch")
```

```
time.sleep(1)
```

```
st.rerun()
```





```

warnings.warn(
/home/student/venv/lib/python3.13/site-packages/sklearn/base.py:463: InconsistentVersionWarning: Trying to unpickle estimator StandardScaler from version 1.6.1 when using version 1.8.0. This might lead to br
aking code or invalid results. Use at your own risk. For more info please refer to:
https://scikit-learn.org/stable/model_persistence.html#security-maintainability-limitations
warnings.warn(
/home/student/realtime.py:63: RuntimeWarning: This channel is already in use, continuing anyway. Use GPIO.setwarnings(False) to disable warnings.
GPIO.setup(22, GPIO.OUT)
System started...
Curr:0.76 Temp:35.88 Vib:10.61
Curr:0.69 Temp:35.88 Vib:12.96
Curr:0.69 Temp:35.88 Vib:13.98
Curr:0.69 Temp:35.94 Vib:12.86
Curr:0.76 Temp:35.94 Vib:11.88
Curr:0.76 Temp:35.88 Vib:11.77
Curr:0.76 Temp:35.88 Vib:11.59
Curr:0.76 Temp:35.88 Vib:12.24
Curr:0.69 Temp:35.88 Vib:13.47
Curr:0.76 Temp:35.94 Vib:13.41
Curr:0.76 Temp:35.88 Vib:15.46
Curr:0.76 Temp:35.88 Vib:10.63
Curr:0.76 Temp:35.88 Vib:12.43
Curr:0.76 Temp:35.88 Vib:12.62
Curr:0.76 Temp:35.88 Vib:9.87
Curr:0.76 Temp:35.94 Vib:21.56
Curr:0.76 Temp:35.88 Vib:15.16
Curr:0.76 Temp:35.94 Vib:13.98
Curr:0.76 Temp:35.88 Vib:13.94
Curr:0.76 Temp:35.88 Vib:11.12
[STATE] NORMAL | Curr:0.74 PeakVib:21.56
Final Prediction: normal
Probabilities: [[0. 0.1 0.22 0.48 0.2]]
Classes: ['highCurrent' 'jam' 'noLoad' 'normal' 'overload']
Curr:0.76 Temp:35.88 Vib:12.86
Curr:0.69 Temp:35.88 Vib:12.65
Curr:0.76 Temp:35.88 Vib:18.29
Curr:0.97 Temp:35.94 Vib:11.83
Curr:0.97 Temp:35.88 Vib:11.93
Curr:1.84 Temp:35.88 Vib:11.91
Curr:0.97 Temp:35.88 Vib:11.98
Curr:0.97 Temp:35.88 Vib:11.89
Curr:0.97 Temp:35.94 Vib:11.72
Curr:0.97 Temp:35.94 Vib:12.19
Curr:0.97 Temp:35.88 Vib:11.97
Curr:1.84 Temp:35.94 Vib:11.78
Curr:1.84 Temp:35.94 Vib:11.84
Curr:0.98 Temp:35.88 Vib:11.79
Curr:0.97 Temp:35.88 Vib:11.92
Curr:0.97 Temp:35.88 Vib:10.89
Curr:0.97 Temp:35.88 Vib:11.82
▲JAM detected! Turning motor OFF
[STATE] JAM | Curr:0.94 PeakVib:18.29
Final Prediction: jam
Probabilities: [[0. 0.1 0.22 0.47 0.21]]
Classes: ['highCurrent' 'jam' 'noLoad' 'normal' 'overload']
Fault detected! Turning motor OFF
Curr:0.98 Temp:35.88 Vib:11.89
Curr:0.97 Temp:35.88 Vib:11.99
Curr:0.97 Temp:35.88 Vib:11.81
Curr:0.97 Temp:35.94 Vib:11.88
Curr:1.84 Temp:35.94 Vib:12.68
Curr:0.98 Temp:35.94 Vib:11.88
Curr:0.83 Temp:35.88 Vib:12.02

```

Appendix B — Sensor Data Sheet References

The following data sheets provide complete technical specifications for the key sensors and modules used in this project. Students are encouraged to consult the official data sheets for detailed electrical characteristics, application circuit recommendations, and communication protocol timing diagrams.

- ACS712 Current Sensor Family Data Sheet: Allegro MicroSystems LLC. ACS712ELC-20A — Fully Integrated, Hall Effect-Based Linear Current Sensor with 2.1 kVRMS Voltage Isolation and a Low-Resistance Current Conductor. Available at: www.allegromicro.com
- DS18B20 Temperature Sensor Data Sheet: Maxim Integrated (now Analog Devices). DS18B20 Programmable Resolution 1-Wire Digital Thermometer. Document Rev. 6. Available at: www.analog.com/en/products/ds18b20.html
- MPU-6050 Product Specification: TDK InvenSense. MPU-6000 and MPU-6050 Product Specification, Revision 3.4. Document PS-MPU-6000A-00. Available at: invensense.tdk.com
- PCF8591 Data Sheet: NXP Semiconductors. PCF8591 — 8-bit A/D and D/A Converter. Document NXP PCF8591. Available at: www.nxp.com/products/analog-and-mixed-signal/pcf8591
- Raspberry Pi 4 Product Brief and Documentation: Raspberry Pi Foundation. Raspberry Pi 4 Hardware Documentation and Pinout Reference. Available at: www.raspberrypi.com/documentation

References

Books and Textbooks

1. [1] Nagrath, I. J., and Kothari, D. P. (2010). Electric Machines (4th ed.). Tata McGraw-Hill Publishing Company, New Delhi.
2. [2] Monk, S. (2021). Programming the Raspberry Pi: Getting Started with Python (3rd ed.). McGraw-Hill Education, New York.
3. [3] Rajaraman, V. (2018). Internet of Things: Enabling Technologies, Applications and Security. PHI Learning Pvt. Ltd., New Delhi.
4. [4] Goodfellow, I., Bengio, Y., and Courville, A. (2016). Deep Learning. MIT Press, Cambridge, MA.

Journal Articles and Technical Papers

5. [5] Nandi, S., Toliyat, H. A., and Li, X. (2005). Condition Monitoring and Fault Diagnosis of Electrical Motors — A Review. IEEE Transactions on Energy Conversion, 20(4), 719–729.
6. [6] Lee, J., Bagheri, B., and Kao, H. A. (2015). A Cyber-Physical Systems Architecture for Industry 4.0-Based Manufacturing Systems. Manufacturing Letters, 3, 18–23.
7. [7] Susto, G. A., Schirru, A., Pampuri, S., McLoone, S., and Beghi, A. (2015). Machine Learning for Predictive Maintenance: A Multiple Classifier Approach. IEEE Transactions on Industrial Informatics, 11(3), 812–820.
8. [8] Atzori, L., Iera, A., and Morabito, G. (2010). The Internet of Things: A Survey. Computer Networks, 54(15), 2787–2805.
9. [9] Patel, R. K., and Bhatt, M. (2020). IoT-Based Real-Time Monitoring System for Industrial Machine Fault Detection Using Machine Learning. Journal of Engineering Research and Application, 10(5), 45–52.

Online Resources and Data Sheets

10. [10] Allegro MicroSystems. (2018). ACS712 Data Sheet. Retrieved from <https://www.allegromicro.com>
11. [11] Analog Devices / Maxim Integrated. (2019). DS18B20 Data Sheet (Rev. 6). Retrieved from <https://www.analog.com>

-
12. [12] TDK InvenSense. (2013). MPU-6050 Product Specification Rev. 3.4. Retrieved from <https://invensense.tdk.com>
 13. [13] NXP Semiconductors. (2013). PCF8591 Data Sheet. Retrieved from <https://www.nxp.com>
 14. [14] Raspberry Pi Foundation. (2023). Raspberry Pi 5 Documentation. Retrieved from <https://www.raspberrypi.com/documentation>
 15. [15] Python Software Foundation. (2024). Python 3 Documentation. Retrieved from <https://docs.python.org/3>
 16. [16] Scikit-learn Developers. (2024). Scikit-learn: Machine Learning in Python. Retrieved from <https://scikit-learn.org>
 17. [17] smbus2 Library Documentation. Retrieved from <https://pypi.org/project/smbus2>
 18. [18] RPi.GPIO Library Documentation. Retrieved from <https://pypi.org/project/RPi.GPIO>